

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»
Тема: Методы приближенного вычисления интегралов

Студент гр. 4341

Малышев К.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью лабораторной работы является изучение и реализация итерационного метода Якоби для решения систем линейных алгебраических уравнений, а также исследование сходимости, точности и эффективности метода на матрицах с диагональным преобладанием.

Задание.

Вариант 12:
$$f(x) = \int_1^2 \ln x (x + 1)^{-1} dx$$

1) Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.

2) Составить программу-функцию для вычисления подынтегральной функции.

3) Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы.

4) Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.

5) Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Варианты заданий приведены в таблице 4.1 ($\varepsilon = 0.01; 0.001; 0.0001$).

Описание метода.

Метод средних прямоугольников

Метод средних прямоугольников заключается в разбиении интервала интегрирования $[a, b]$ на n равных частей длиной $h = \frac{b - a}{n}$

На каждом элементарном отрезке значение функции заменяется её значением в середине отрезка.

Площадь под графиком функции приближённо вычисляется как сумма площадей прямоугольников.

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f(a + h(i + \frac{1}{2}))$$

Метод обладает сравнительно высокой точностью по сравнению с обычным методом левых или правых прямоугольников.

Метод трапеций

Метод трапеций основан на замене функции на каждом отрезке линейной функцией (прямой), соединяющей соседние точки графика.

Таким образом, площадь под кривой аппроксимируется суммой площадей трапеций.

$$\int_a^b f(x) dx \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih) \right)$$

Данный метод даёт более точный результат, чем метод прямоугольников, особенно для функций, близких к линейным.

Метод Симпсона.

Метод Симпсона (метод парабол) заключается в аппроксимации функции на каждом парном интервале квадратичной функцией (параболой).

Для применения метода необходимо, чтобы число разбиений n было чётным.

$$\int_a^b f(x) dx \approx \frac{h}{3} [f(a) + f(b) + 4 \sum_{i \text{ odd}} f(x_i) + 2 \sum_{i \text{ even}} f(x_i)]$$

Метод Симпсона обеспечивает более высокую точность по сравнению с методами прямоугольников и трапеций, так как учитывает кривизну функции.

Для контроля точности численного интегрирования используется правило Рунге.

Оно основано на сравнении результатов вычислений при двух разных шагах разбиения.

Пусть:

I_n - значение интеграла при разбиении на n частей,

I_{2n} - значение при удвоенном числе разбиений.

Тогда оценка погрешности вычисляется по формуле:

$$R = \frac{|I_{2n} - I_n|}{2^p - 1}$$

где p — порядок метода:

$p = 2$ — для метода прямоугольников и трапеций,

$p = 4$ — для метода Симпсона.

Алгоритм применения правила Рунге

1. Задаётся начальное число разбиений n .
2. Вычисляются значения интеграла I_n и I_{2n} .
3. Определяется оценка погрешности R .
4. Если $R > \varepsilon$, число разбиений удваивается: $n := 2n$.
5. Процесс повторяется до тех пор, пока не выполнится условие:
 $R < \varepsilon$

Разработка программы.

Программа, реализующая методы приближённого вычисления определённого интеграла, была написана на языке программирования C++.

В соответствии с заданием, были реализованы три основные функции:

- `double integral_rectangle(const function<double(double)>& f, double a, double b, int n)` — метод средних прямоугольников;
- `double integral_trapezoidal(const function<double(double)>& f, double a, double b, int n)` — метод трапеций;
- `double integral_simpson(const function<double(double)>& f, double a, double b, int n)` — метод Симпсона.

Каждая функция принимает следующие параметры:

- `f` — подынтегральная функция, передаваемая в виде функционального объекта `(std::function<double(double)>)`;
- `a` — нижний предел интегрирования;
- `b` — верхний предел интегрирования;
- `n` — количество разбиений интервала `[a,b]`.

Функции возвращают приближённое значение определённого интеграла.

Результаты вычислений.

Тест:

$$f(x) = \int_1^2 \ln x (x + 1)^{-1} dx$$

$\epsilon \in \{0.01, 0.001, 0.0001\}$

Метод	ϵ	n	Значение	Погрешность
Прямоугольник	0.01	4	0.148279	0.00101902
Трапеция		4	0.145096	0.00208008
Симпсон		4	0.147176	3.62274e-05
Прямоугольник	0.001	8	0.147487	0.000263858
Трапеция		8	0.146687	0.000530534
Симпсон		4	0.147176	3.62274e-05
Прямоугольник	0.0001	16	0.147287	6.65791e-05
Трапеция		32	0.147187	3.33793e-05
Симпсон		4	0.147176	3.62274e-05

Вывод.

В ходе лабораторной работы были реализованы методы прямоугольников, трапеций и Симпсона для численного вычисления определённого интеграла с заданной точностью.

Для оценки точности использовалось правило Рунге, позволяющее контролировать погрешность без знания точного значения интеграла.

По результатам вычислений установлено:

- при увеличении точности ($\epsilon = 0.01 \rightarrow 0.0001$) для методов прямоугольников и трапеций требуется увеличение числа разбиений n;

- метод Симпсона обеспечивает высокую точность уже при малом числе разбиений ($n = 4$) и практически не требует увеличения n .

Также видно, что все методы сходятся к близкому значению интеграла: $I \approx 0.1472$

Приложение А

Исходный код программы

```
#include <iostream>
#include <functional>
#include <cmath>

using namespace std;

// Подынтегральная функция
double f(double x) {
    return log(x) / (x + 1);
}

// 1. Прямоугольники
double integral_rectangle(const function<double(double)>& f, double a,
double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;

    for (int i = 0; i < n; ++i) {
        double x_mid = a + h * (i + 0.5);
        sum += f(x_mid);
    }

    return sum * h;
}

// 2. Трапеции
double integral_trapezoidal(const function<double(double)>& f, double
a, double b, int n) {
    double h = (b - a) / n;
    double sum = (f(a) + f(b)) / 2.0;

    for (int i = 1; i < n; ++i) {
        sum += f(a + i * h);
    }

    return sum * h;
}

// 3. Симпсон
double integral_simpson(const function<double(double)>& f, double a,
double b, int n) {
    if (n % 2 != 0) n++;

    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; ++i) {
        double x = a + i * h;
        sum += (i % 2 == 0 ? 2 : 4) * f(x);
    }

    return sum * h / 3.0;
}
```



```

}

// Метод с автоматическим подбором n
void runge_method(const function<double(const
function<double(double)>&, double, double, int)>& method,
const function<double(double)>& f,
double a, double b, double eps, int p,
const string& name) {

    int n = 2;
    double I_n, I_2n, R;
    int iter = 0;

    cout << name << endl;
    cout << "Итерации:" << endl;

    do {
        I_n = method(f, a, b, n);
        I_2n = method(f, a, b, 2 * n);

        R = fabs(I_2n - I_n) / (pow(2, p) - 1);

        iter++;

        cout << "Итерация " << iter
            << ": n = " << n
            << ", I_n = " << I_n
            << ", I_2n = " << I_2n
            << ", R = " << R << endl;

        n *= 2;

    } while (R > eps);

    cout << "-----" << endl;
    cout << "ИТОГ:" << endl;
    cout << "Интеграл ≈ " << I_2n << endl;
    cout << "n = " << n << endl;
    cout << "Оценка погрешности = " << R << endl;
    cout << "Количество итераций = " << iter << endl;
    cout << "=====" << endl;
}

int main() {
    double a = 1.0;
    double b = 2.0;

    double eps_values[] = {1e-6};

    for (double eps : eps_values) {
        cout << "Точность eps = " << eps << endl;
        cout << "=====" << endl;

        runge_method(integral_rectangle, f, a, b, eps, 2,
"Прямоугольники");
        runge_method(integral_trapezoidal, f, a, b, eps, 2,
"Трапеции");
        runge_method(integral_simpson, f, a, b, eps, 4, "Симпсон");
    }
}

```

```
    return 0;  
}
```